编译原理
18. 循环优化

rainoftime.github.io
浙江大学
计算机科学与技术学院

Content

Loop Optimizations

- Loops are pervasive in computer programs
- A great proportion of the execution time of a typical program is spent in loops.
- So we want techniques to improve loops!

- Loop invariant hoisting
- Induction variable reduction
- Loop unrolling
- Loop fusion
- Loop fission
- Loop interchange
- Loop peeling
- Loop tiling
- Loop parallelization
- …

Example: Loop Invariant Hoisting

- **t = a + b** is a loop invariant

```
L0:   t := 0

L1:   i := i + 1
      t := a + b
      *i := t
      if i<N goto L1 else L2

L2:   x := t
```

Example: Loop Invariant Hoisting

- We can move **t = a + b** for optimization

```
L0:   t := 0
      t := a + b


L1:   i := i + 1

      *i := t
      if i<N goto L1 else L2

 L2:   x := t
```

Outline

1. Loops and Dominators
2. Loop Invariant Hoisting
3. Induction Variables
4. Array Bounds Check
5. Loop Unrolling

Part II
(不考)

1. Loops and Dominators

- Loops
- Finding Loops
- Loop Preheader

What is a Loop?

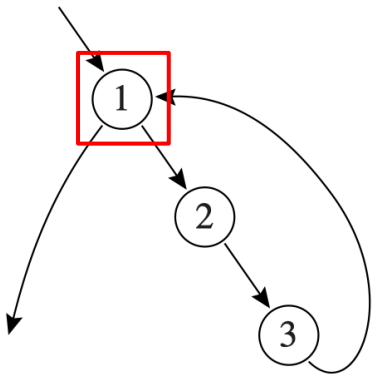
- A **loop** in a control-flow graph is a set of nodes S including a header node h such that:
  1. From any node in S there is a **path** leading to h.
  2. There is a **path** from h to any node in S.
  3. There is **no edge** from any node outside S to any node in S other than h.

The definition differs from the dictionary definition.
It's specifically designed for compiler optimization purposes.

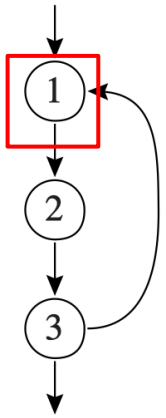
What is a Loop?

- A **loop entry** node is one with predecessor outside the loop.
- A **loop exit** node is one with a successor outside the loop.

  - *h* -> *S*
  - *S* -> *h*
  - no other node -> *S*



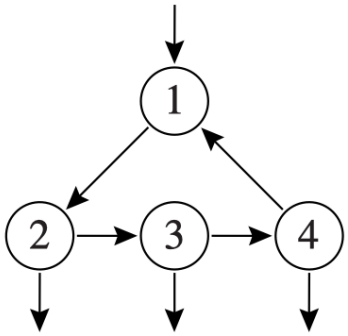
(a)



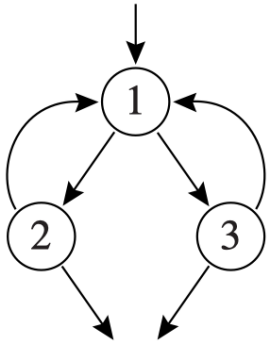
(b)

Only **one entry**, but may have **multiple exists**

头节点是循环唯一入口



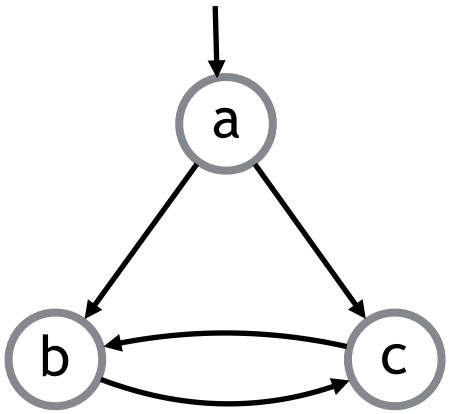
(c)



(d)

Example: Non-Loops

- a can't be the loop header
  - No path from b to a or c to a
- b can't be the loop header
  - Has outside edge from a
- c can't be the loop header
  - Has outside edge from a



- So no "loop"... (But clearly, there is a cycle!!)

Why Defining Loops this Way?

- Loop header gives us a "handle" for the loop
  - Good spot for placing loop optimizations (e.g., hoisting invariant code)
- Structured control-flow only produces **reducible graph**
  - Many loop optimizations depend upon having reducible graphs

Reducible Graphs

- All cycles in the graph form loops according to our def
- Most high-level languages produce reducible graphs
- E.g., Java produces only reducible graphs

Irreducible Graphs

- Contain cycles that aren't loops by our definition
- Can be produced by unstructured control flow
- C/C++ with goto statements can produce irreducible graphs

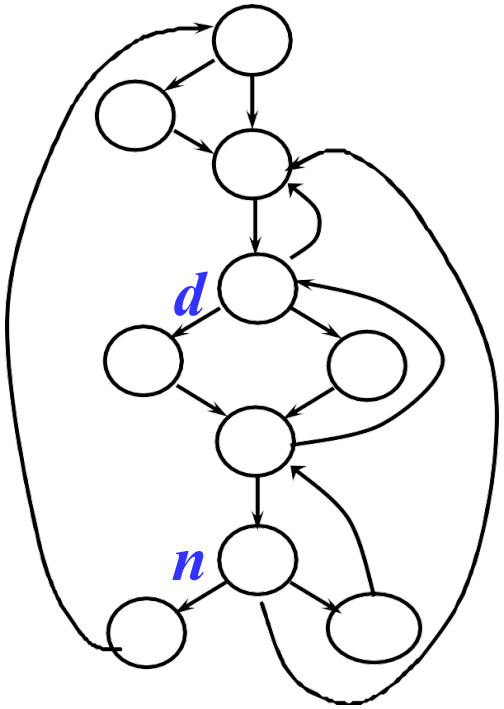
1. Loops and Dominators

- Loops
- Finding Loops
  - Dominance
  - Loop
- Loop Preheader

Dominator (支配结点)

- A node *d* **dominates** a node *n* (***d* dom *n*)** if every path of directed edges from the start node to *n* must go through *d*.

  d dom n: 从流图的入口结点s_0到结点*n*的每条路径都经过节点*d*



- Some properties of dominator
  - Every node dominates itself
  - *n* can have more than one dominators

Finding Dominators

- Let D[n] be the set of nodes that dominate *n*

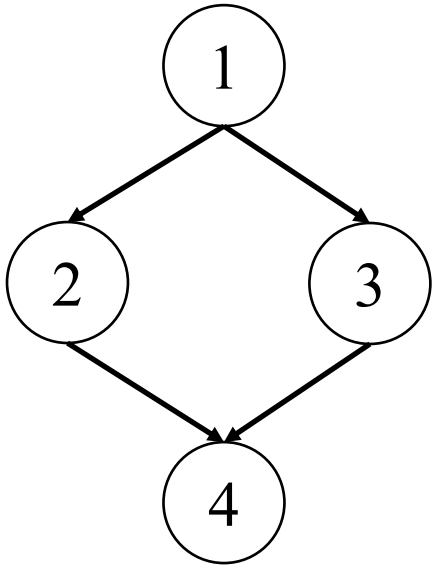
$$D[s_0] = \{s_0\} \qquad D[n] = \{n\} \cup \left(\bigcap_{p \in \text{pred}[n]} D[p] \right) \qquad \text{for } n \neq s_0$$

1. Start off assuming D[n] is all nodes, except for the start node (which is only dominated by itself)

2. Iteratively update D[n] based on **predecessors** until reaching a fixed point

Example: Finding Dominators

$$D[s_0] = \{s_0\} \qquad D[n] = \{n\} \cup \left(\bigcap_{p \in \text{pred}[n]} D[p] \right) \qquad \text{for } n \neq s_0$$

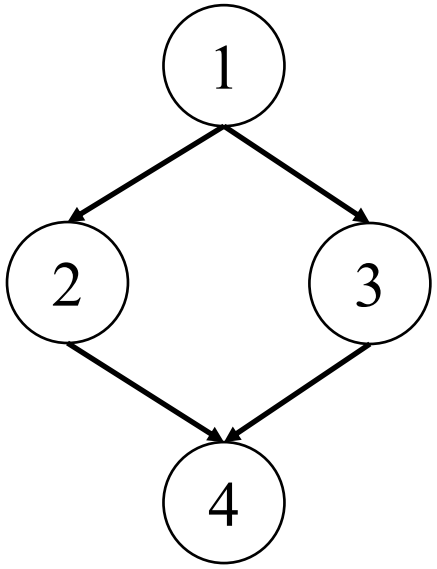


Initialization:

- D[1] = {1}
  (start node is only dominated by itself)
- D[2] = D[3] = D[4] = {1,2,3,4}
  (all nodes in the graph)

Example: Finding Dominators

$$D[s_0] = \{s_0\} \qquad D[n] = \{n\} \cup \left(\bigcap_{p \in \text{pred}[n]} D[p] \right) \qquad \text{for } n \neq s_0$$

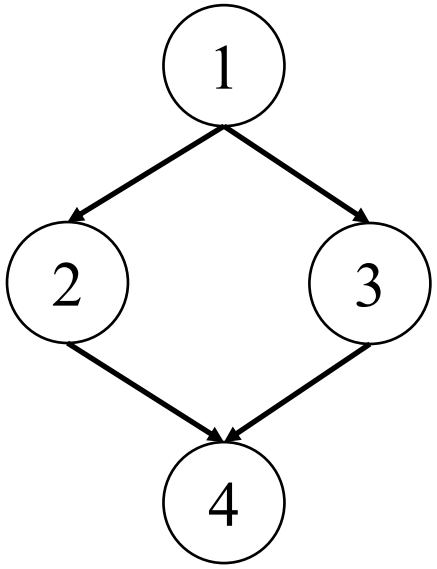


Iteration 1:

- D[2] = {2} ∪ (D[1]) = {2} ∪ {1} = {1,2}
  Changed!
- D[3] = {3} ∪ (D[1]) = {3} ∪ {1} = {1,3}
  Changed!
- D[4] = {4} ∪ (D[2] ∩ D[3]) =
  {4} ∪ ({1,2} ∩ {1,3}) = {4} ∪ {1} = {1,4}
  Changed!

Example: Finding Dominators

$$D[s_0] = \{s_0\} \qquad D[n] = \{n\} \cup \left(\bigcap_{p \in \text{pred}[n]} D[p] \right) \qquad \text{for } n \neq s_0$$



Iteration 2:

- D[2] = {2} ∪ (D[1]) = {2} ∪ {1} = **{1,2}**
  No change
- D[3] = {3} ∪ (D[1]) = {3} ∪ {1} = **{1,3}**
  No change
- D[4] = {4} ∪ (D[2] ∩ D[3]) =
  {4} ∪ ({1,2} ∩ {1,3}) = {4} ∪ {1} = **{1,4}**
  No change

Immediate Dominators (直接支配结点)

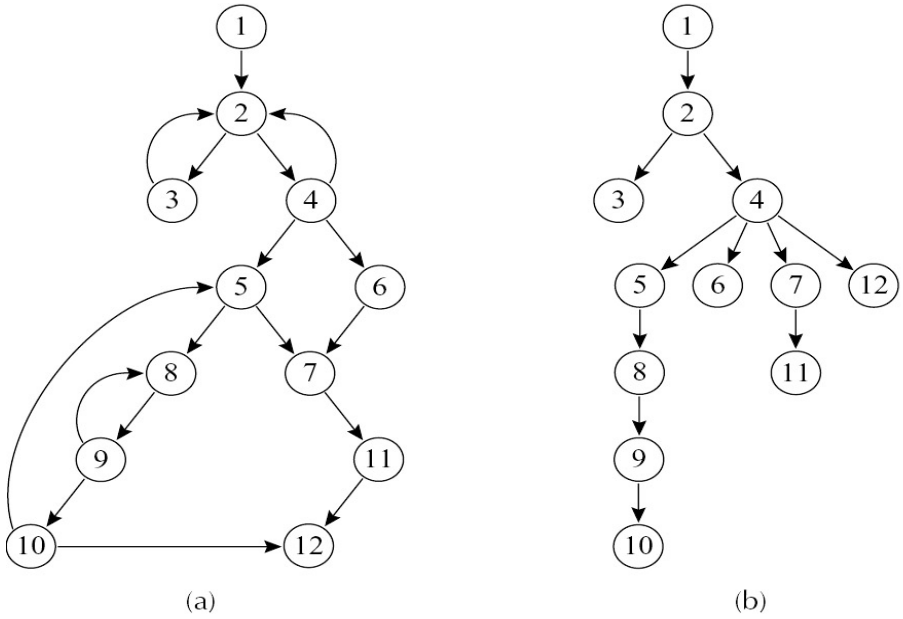
- Every node *n* (except s_0) has exactly one *immediate dominator, idom(n), such that:*
  - idom(n) is not the same node n
  - idom(n) dominates n, and
  - idom(n) does not dominate any other dominator of n

从入口结点到达n的任何路径 (不含n) 中，
idom是路径中最后一个支配n的结点

Theorem: Suppose *d* and *e* dominate *n*, it must be that either *d* dominates *e* or *e* dominates *d*.

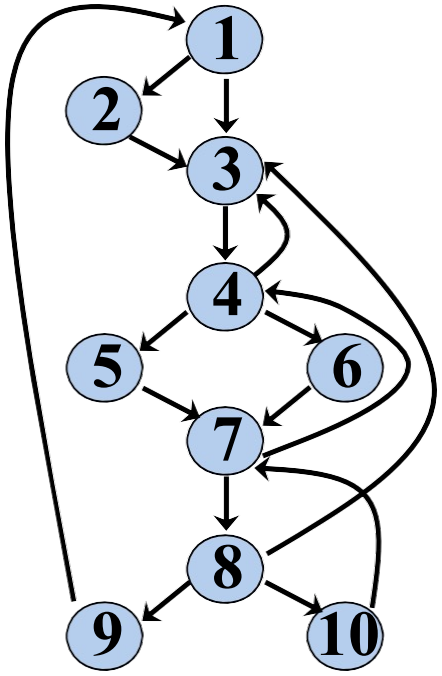
Dominator Tree (支配结点树)

- Instead of keeping the full set of dominators for each node, we can build a **dominator tree**
- **Dominator tree**: for every node *n*, there is an edge from *idom(n)* to n.



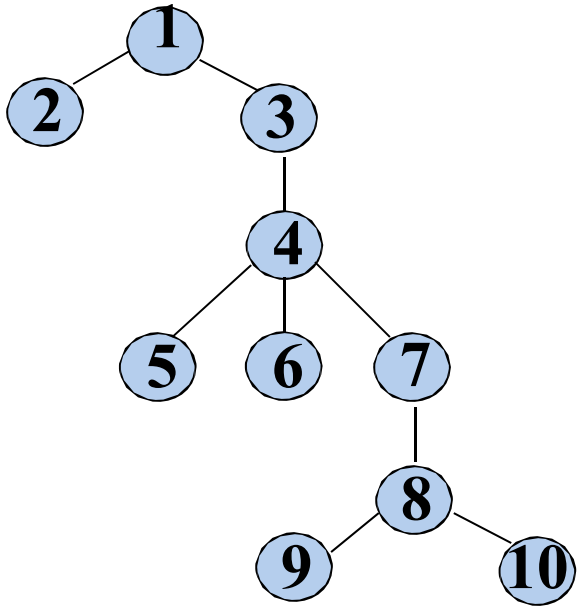
在Dom tree中,节点的父节点是其idom，祖先是其所有支配者

Example: Dominator Tree



支配结点	支配对象
1	1 ~ 10
2	2
3	3 ~ 10
4	4 ~ 10
5	5
6	6
7	7 ~ 10
8	8 ~ 10
9	9
10	10

- **支配结点树 (*Dominator Tree*)**



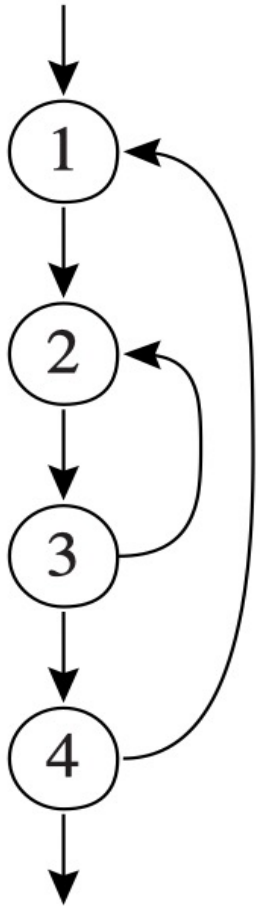
1. Loops and Dominators

- Loops
- Finding Loops
  - Dominance
  - Loop
- Loop Preheader

Natural Loops (自然循环)

- **Back Edge**: an edge from n to h when h **dominates** n (h dom n)
- The **natural loop** of a back edge $n \rightarrow h$ is the set of nodes x such that
  1. h dominates x and
  2. there is a path from x to n not containing h

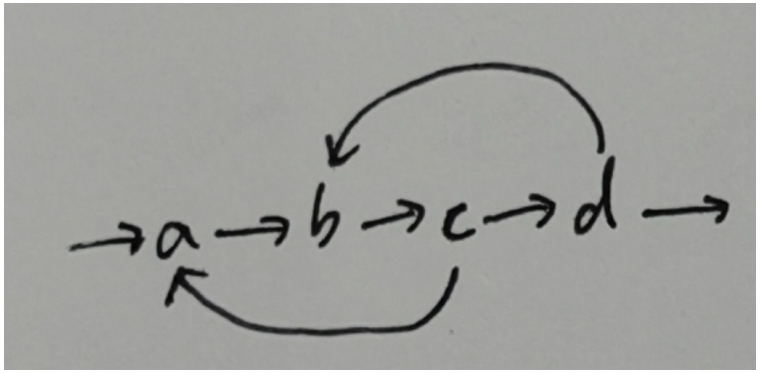
The loop header is h



Identifying Loops from Back Edges

- 在右下图中, {a, b, c}是一个natural loop吗？

构造回边 x→h 的自然环：
1. Loop ← {h}
2. WorkList ← {x}
3. 当 WorkList 非空时：
   - 取出节点 p
   - 如果 p ∉ Loop：
     - 将 p 加入 Loop
     - 将 p 的所有前驱节点加入 WorkList (但不包括回边 x→h)

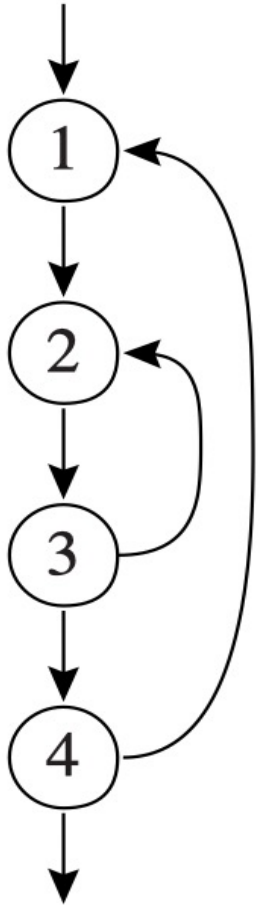


{a,b,c} 无法被某条回边唯一地生成；真正的自然环是
- {b,c,d}　（头结点 h=b，对应回边 d → b）
- {a,b,c,d}（头结点 h=a，对应回边 c → a）

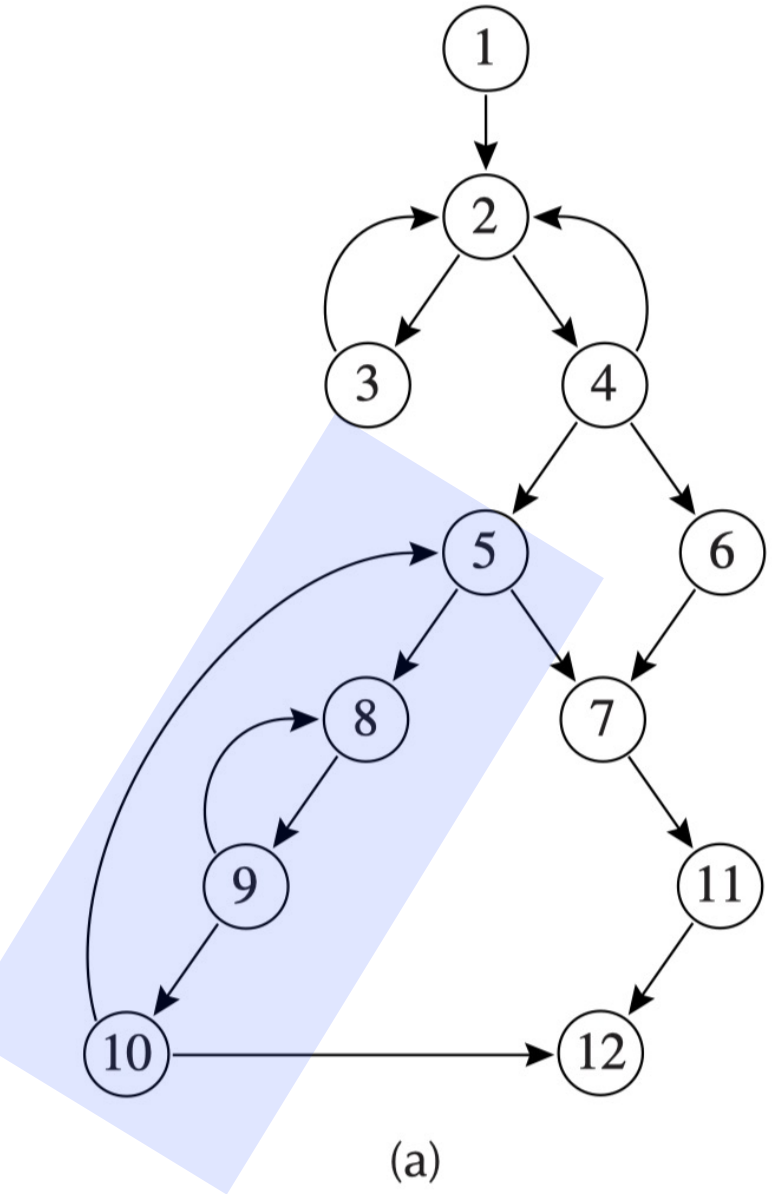
Natural Loops (自然循环)

- 从编译器的角度看，循环在代码中以什么形式出现并不重要，重要的是它是否具有易于优化的性质

- **自然循环满足以下性质**
  - 有唯一的入口结点，称为首结点(*header*)。首结点支配循环中的所有结点
  - 循环中至少有一条返回首结点的路径

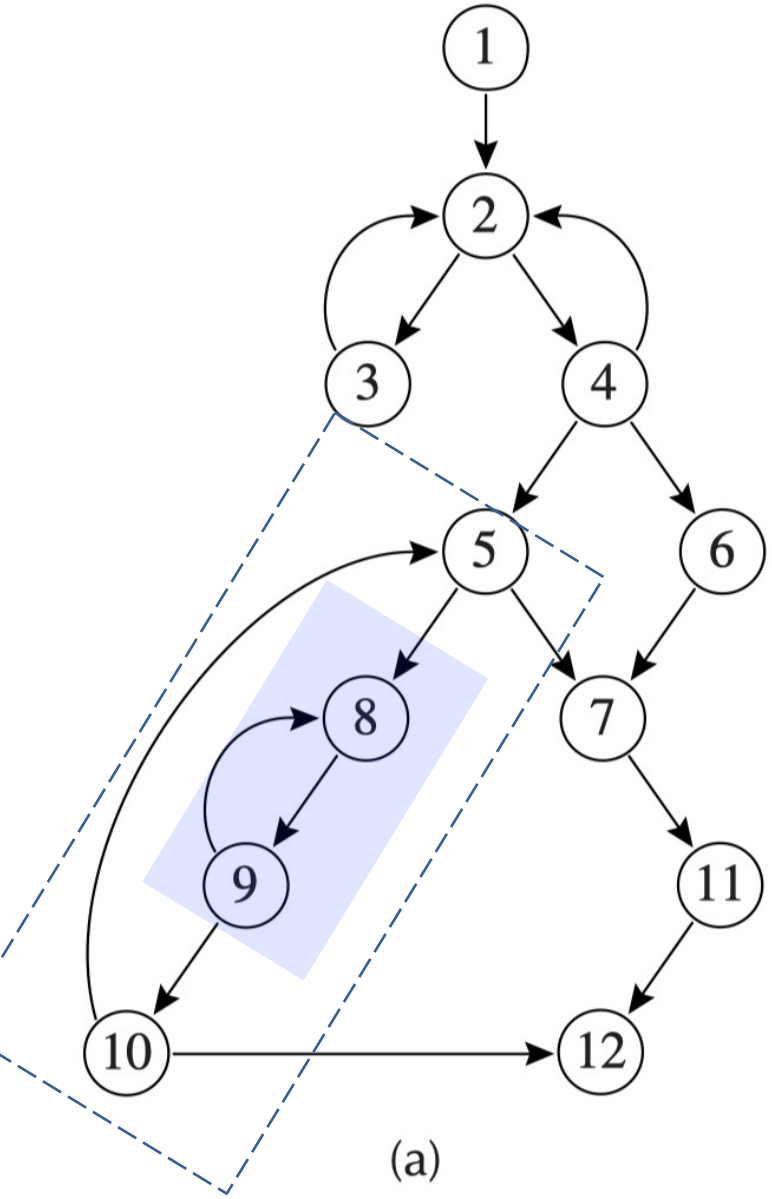


Example: Natural Loops



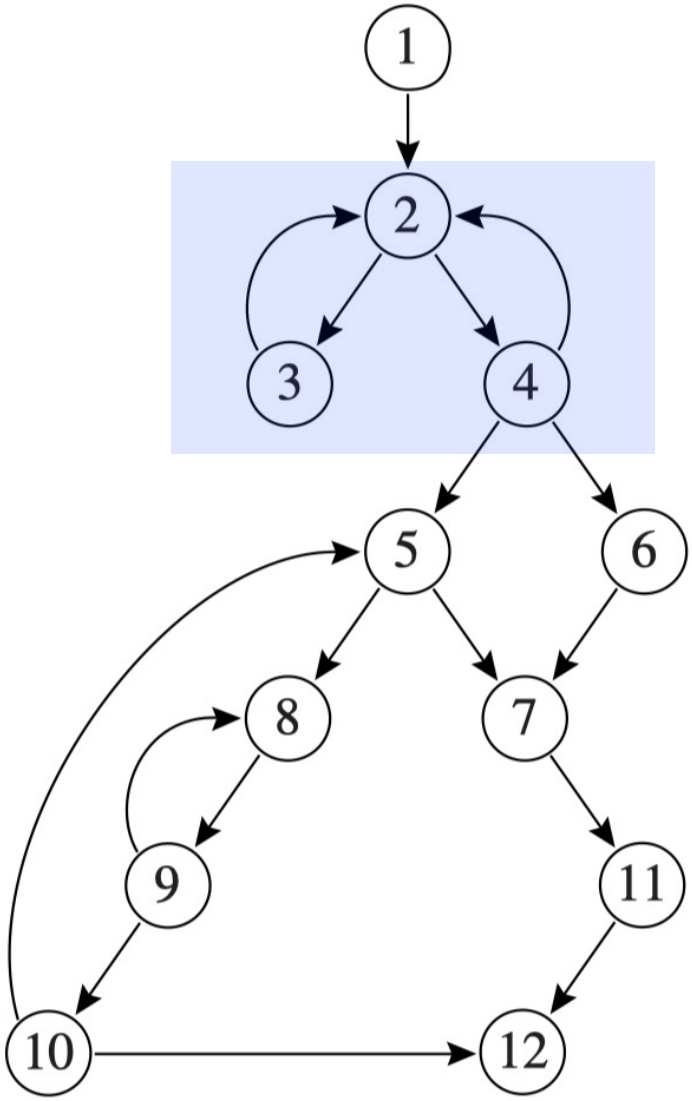
- **The natural loop of the back edge 10 -> 5 includes nodes 5, 8, 9, 10**

Example: Natural Loops



- The natural loop of the back edge 10 -> 5 includes nodes 5, 8, 9, 10
- **The loop 8, 9 is nested within the above loop**

Example: Natural Loops



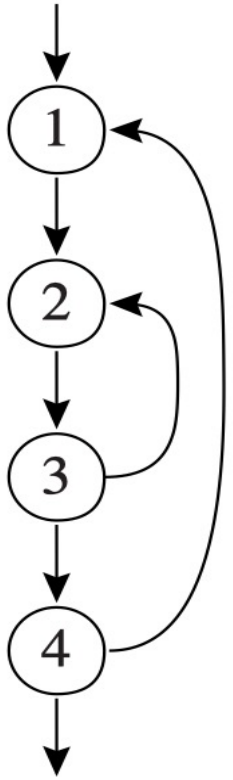
(a)

- The natural loop of the back edge 10 -> 5 includes nodes 5, 8, 9, 10
- The loop 8, 9 is nested within the above loop
- **A node *h* can be the header of more than one loop**
  - 3->2 => {3, 2}
  - 4->2 => {4, 2}

Nested Loop

- If loops A and B have distinct headers and all nodes in B are in A (i.e., B⊆A), then we say B is nested within A
  - B is an inner loop

- 最内循环 (*Innermost Loops*): 不包含其它循环的循环

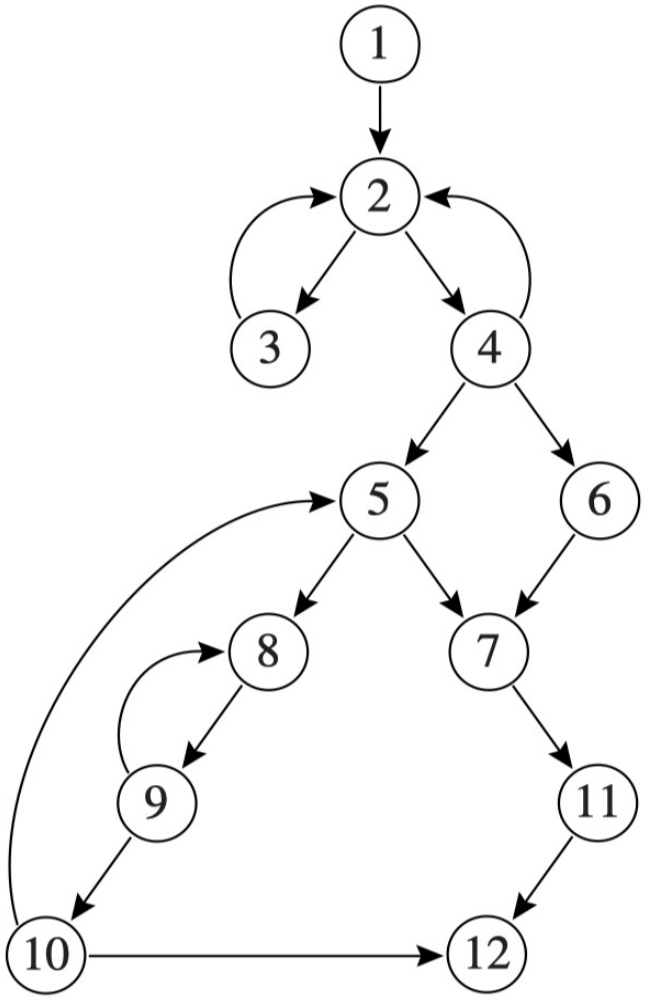


定理: 除非两个自然循环的首结点相同，否则，它们或者互不相交, 或者一个完全包含(嵌入)在另外一个里面

Loop-Nest Tree

We can construct a **loop-nest tree** of loops in a program:

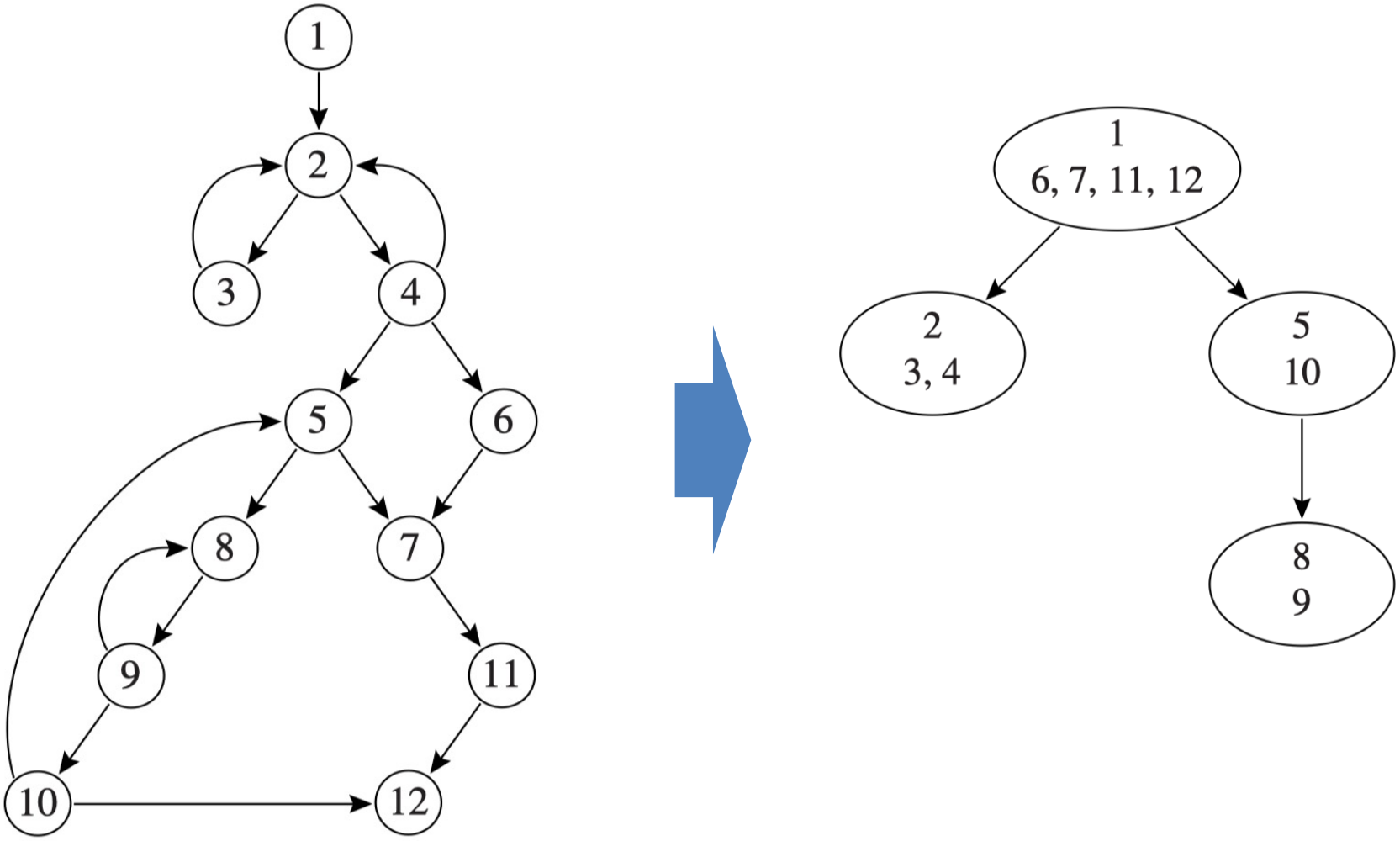
1. Compute dominators of the flow graph G
2. Construct the dominator tree
3. Find all the natural loops, and thus all the loop-header nodes
4. For each loop header *h*, merge all the natural loops of *h* into a single loop, loop[h]
5. Construct the tree of loop headers (and implicitly loops), such that *h1* is above *h2* in the tree if *h2* is in *loop[h1]*



(a)

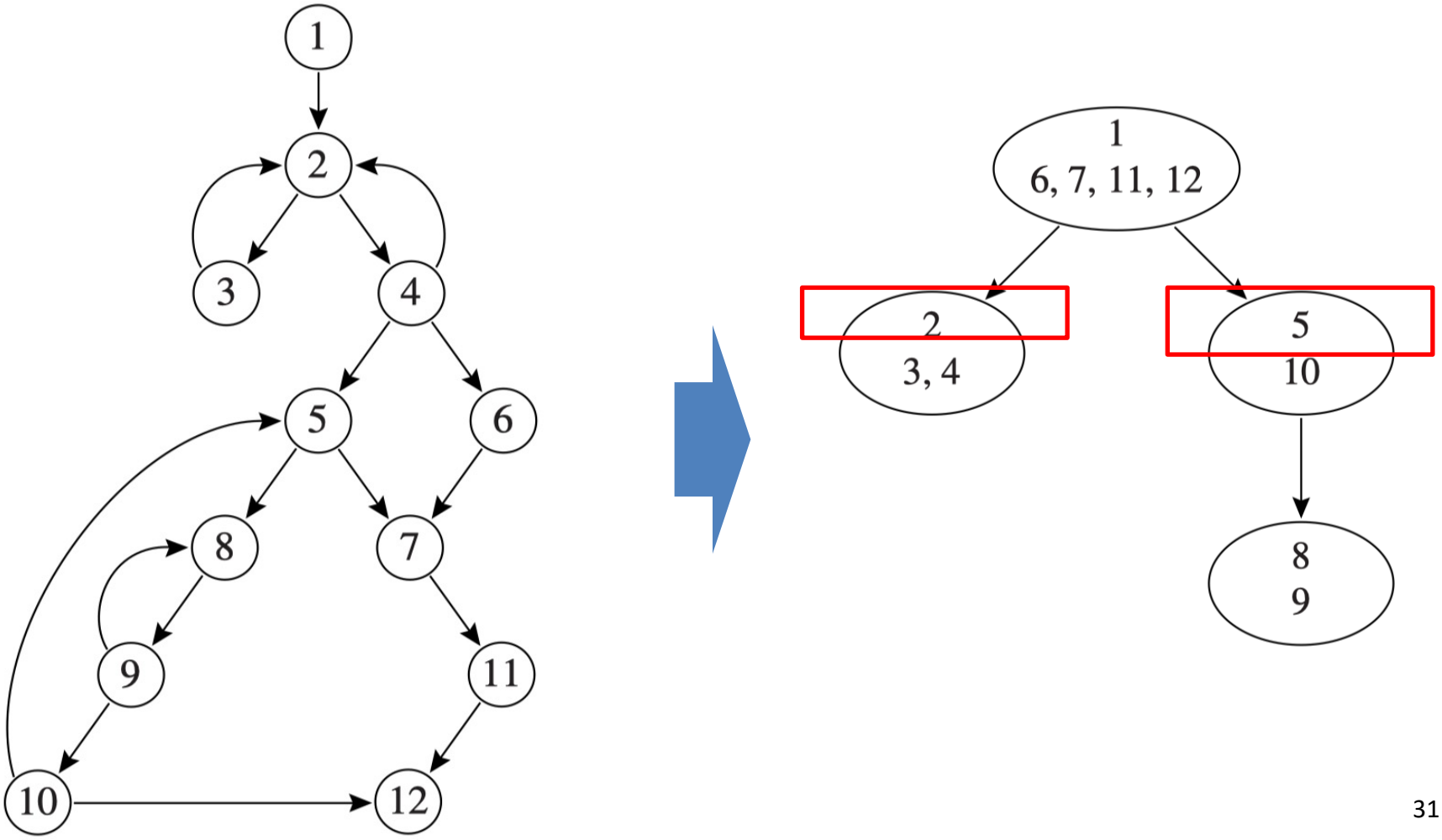
Loop-Nest Tree: Node and Edge

- Each node represents a loop or a collection of loops
- The parent-child relationship indicates nesting



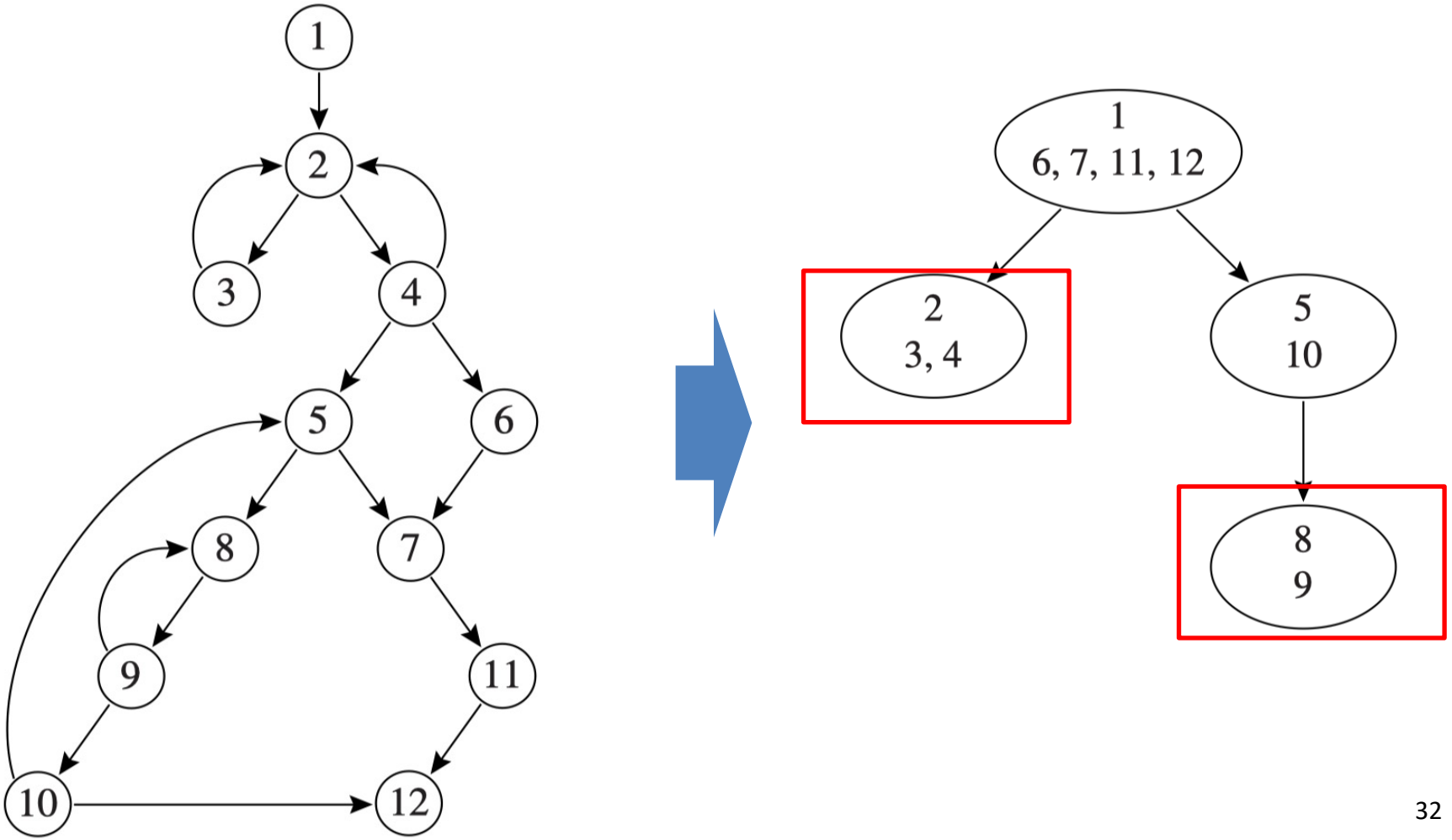
Loop-Nest Tree: Loop Header

- Each loop header is shown in the top half of each oval (nodes 1, 2, 5, 8)



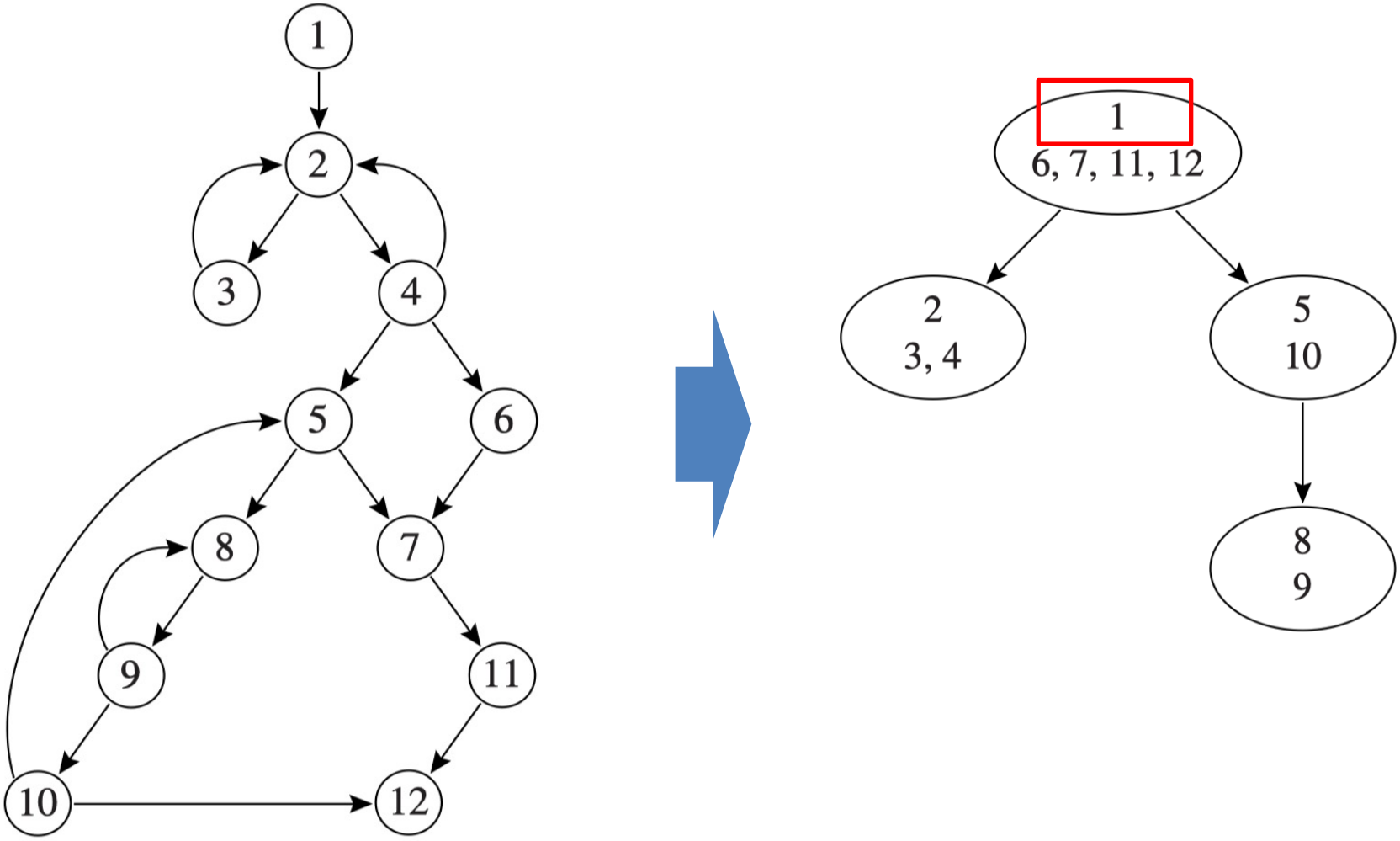
Loop-Nest Tree: Leaf Node

- The leaves of the loop-nest tree are the *innermost loops*



Loop-Nest Tree

- We could say that the entire procedure body is a pseudo-loop that sits at the root of the loop-nest tree.

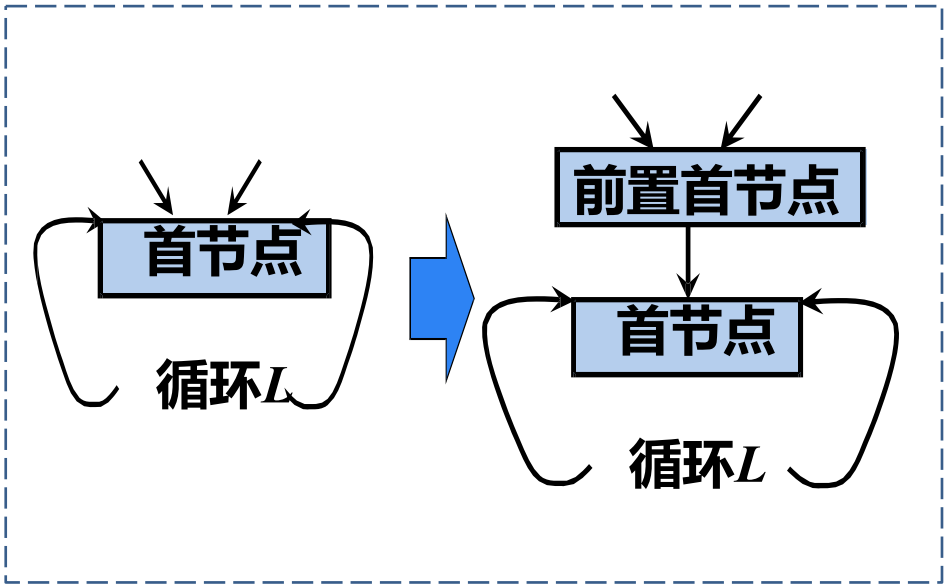


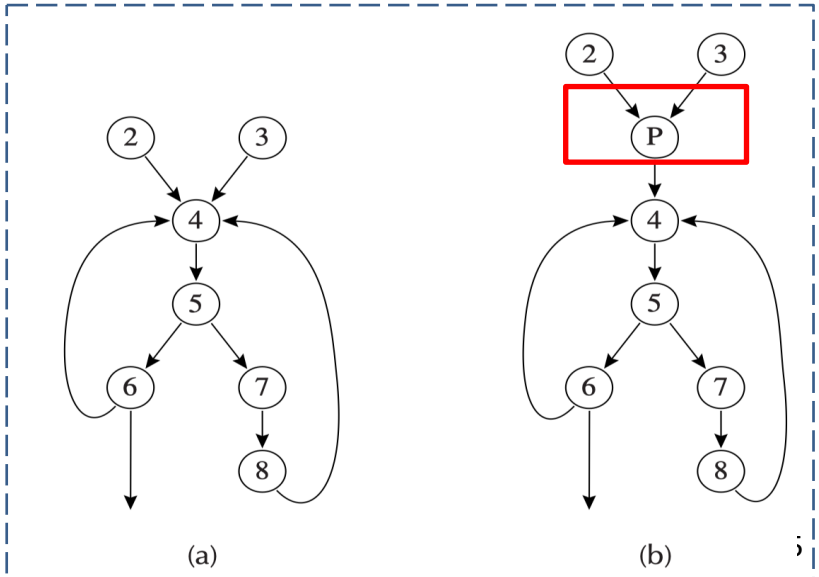
1. Loops and Dominators

- Loops
- Finding Loops
- Loop Preheader

Loop Preheader (前置首结点)

- Many loop optimizations will insert statements immediately before the loop execute
- A pre-header is a basic block inserted immediately before the loop header
  - It has exactly one successor (the loop header)
  - All paths to the loop header pass through the pre-header





2. Loop Invariant Hoisting

- Loop Invariant
- Loop Invariant Hoisting

Loop Invariant Code Motion

- **Idea**: some expressions evaluated in a loop never change; they are loop invariant
  - Move them outside the loop, store result in temporary and just use the temporary in each iteration

- **Step 1 Analysis**: find invariant computations in loop
  - invariant: computes same result each time evaluated

- **Step 2 Transformation**: move them outside loop

Identifying Loop Invariants

An assignment x := v1 op v2 is **invariant** for a loop if for each operand v1 and v2, either

1. The operand is constant, or
2. All of the definitions that reach the assignment are outside the loop, or
3. Only one definition reaches the assignment and it is a loop invariant

Can use an iterative algorithm to compute

Example: Identifying Loop Invariants

```
L0: t := 0
    a := x
L1: i := i + 1
    b := 7
    t := a + b
    *i := t
    if i < N goto L1 else L2
L2: x := t
```

- i := i + 1 Not invariant
  (i changes each iteration)
- b := 7 Invariant
  (7 is a constant)
- t := a + b Invariant (a defined outside loop, b is invariant)
- *i := t Not invariant
  (i changes each iteration)

2. Loop Invariant Hoisting

- Loop Invariant
- Loop Invariant Hoisting

Code Motion (Hoisting)

- **Idea**: some expressions evaluated in a loop never change; they are loop invariant
  - Move them outside the loop, store result in temporary and just use the temporary in each iteration

- **Step 1 Analysis**: find invariant computations in loop
  - invariant: computes same result each time evaluated

- **Step 2 Transformation**: move them outside loop

Example: Valid/Safe Hoisting

- c = a + b is a loop invariant, and we can "safely" perform the invariant hoisting

```
// Before hoisting
L0: t := 0
    a := 5
    b := 7
L1: i := i + 1
    c := a + b // Invariant
    d := i * 2
    if i < N goto L1 else L2
L2: x := c
```

```
// After hoisting
L0: t := 0
    a := 5
    b := 7
    c := a + b // Hoisted
L1: i := i + 1
    d := i * 2
    if i < N goto L1 else L2
L2: x := c
```

Example: Invalid/Unsafe Hoisting

- Can we always move t as long as it is an invariant?

```
// Before hoisting
L0: t := 0
L1: i := i + 1
    *i := t     // Uses t
    t := a + b // Invariant
    if i < N goto L1 else L2
L2: x := t
```

```
// INCORRECT hoisting
L0: t := 0
    t := a + b  // Conflicts with
original value
L1: i := i + 1
    *i := t // Now uses wrong
value of t
    if i < N goto L1 else L2
L2: x := t
```

Just because code is loop invariant doesn't mean we can move it! (should preserve the semantics)

Criteria for Safe Loop Invariant Hoisting

- The **criteria** for safely hoisting invariant assignment

  $d : t \leftarrow a \oplus b$

  1. **Dominance condition:** d dominates all loop exits where t is live-out
  2. **Uniqueness condition:** There is only one definition of t in the loop
  3. **Pre-header condition:** t is not live-out of the loop preheader (that is, t is not live before the loop)

Cond 1: *d* dominates all loop exits where *t* is live-out

- If the definition doesn't dominate an exit where t is used, hoisting could cause a different value to be used at that exit.

```
// Before hoisting
for (int i = 0; i < n; i++) {
    if (cond) {
        t = a + b;
    }
    if (error) {
        break; // Early exit
    }
    use(t);
}
use(t); // t used after loop
```

```
// After hoisting
t = a + b; // Hoisted
for (int i = 0; i < n; i++) {
    if (error) {
        break;
    }
    use(t);
}
use(t); // which t will be used?
```

如果错误地提升了这个赋值，无论如何t都会被赋值为a + b，改变了程序行为。

Cond 2: there is only one definition of *t* in the loop

- Multiple definitions of t in the loop could cause inconsistency if only one definition is hoisted:

```
for (i = 0; i < n; i++) {
    t = a + b;
    use(t);

    // later in the loop
    t = c + d;
    use(t);
}
```

```
t = c + d; // hoist the second def
for (i = 0; i < n; i++) {
    t = a + b;
    use(t);

    // later in the loop
    use(t); // Should use c+d, but now incorrectly uses a+b!
}
```

Cond 3: t is not live-out of the loop preheader

- **(that is, t is not live before the loop)**
- If t is live at loop entry, hoisting would overwrite the earlier value

```
// Before hoisting
L0: t := 0
L1: i := i + 1
      *i := t     // Uses t
      t := a + b  // Invariant
      if i < N goto L1 else L2
L2: x := t
```

```
// INCORRECT hoisting
L0: t := 0
      t := a + b  // Conflicts with
original value
L1: i := i + 1
      *i := t // Now uses wrong
value of t
      if i < N goto L1 else L2
L2: x := t
```

Example: Can We Hoist $d: t \leftarrow a \oplus b$?

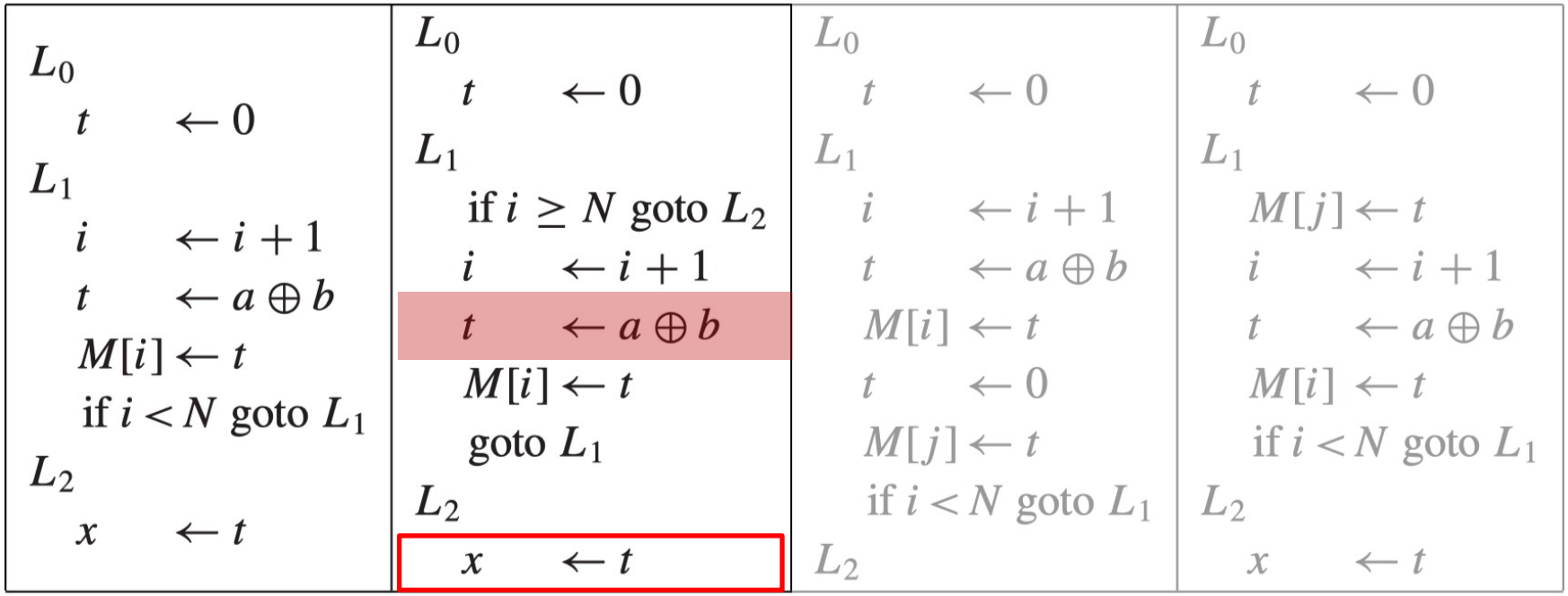
1. d dominates all loop exits where t is live-out
2. and there is only one definition of t in the loop
3. and t is not live-out of the loop preheader

L_0 $t \leftarrow 0$ L_1 $i \leftarrow i+1$ $t \leftarrow a \oplus b$ $M[i] \leftarrow t$ if $i < N$ goto L_1 L_2 $x \leftarrow t$	L_0 $t \leftarrow 0$ L_1 if $i \geq N$ goto L_2 $i \leftarrow i+1$ $t \leftarrow a \oplus b$ $M[i] \leftarrow t$ goto L_1 L_2 $x \leftarrow t$	L_0 $t \leftarrow 0$ L_1 $i \leftarrow i+1$ $t \leftarrow a \oplus b$ $M[i] \leftarrow t$ $t \leftarrow 0$ $M[j] \leftarrow t$ if $i < N$ goto L_1 L_2	L_0 $t \leftarrow 0$ L_1 $M[j] \leftarrow t$ $i \leftarrow i+1$ $t \leftarrow a \oplus b$ $M[i] \leftarrow t$ if $i < N$ goto L_1 L_2 $x \leftarrow t$
correct faster			

Example: Can We Hoist $d: t \leftarrow a \oplus b$?

1. d dominates all loop exits where t is live-out

The original program does not *always* execute $t \leftarrow a \oplus b$, but the transformed program does, producing an incorrect value for x if $i \geq N$ initially

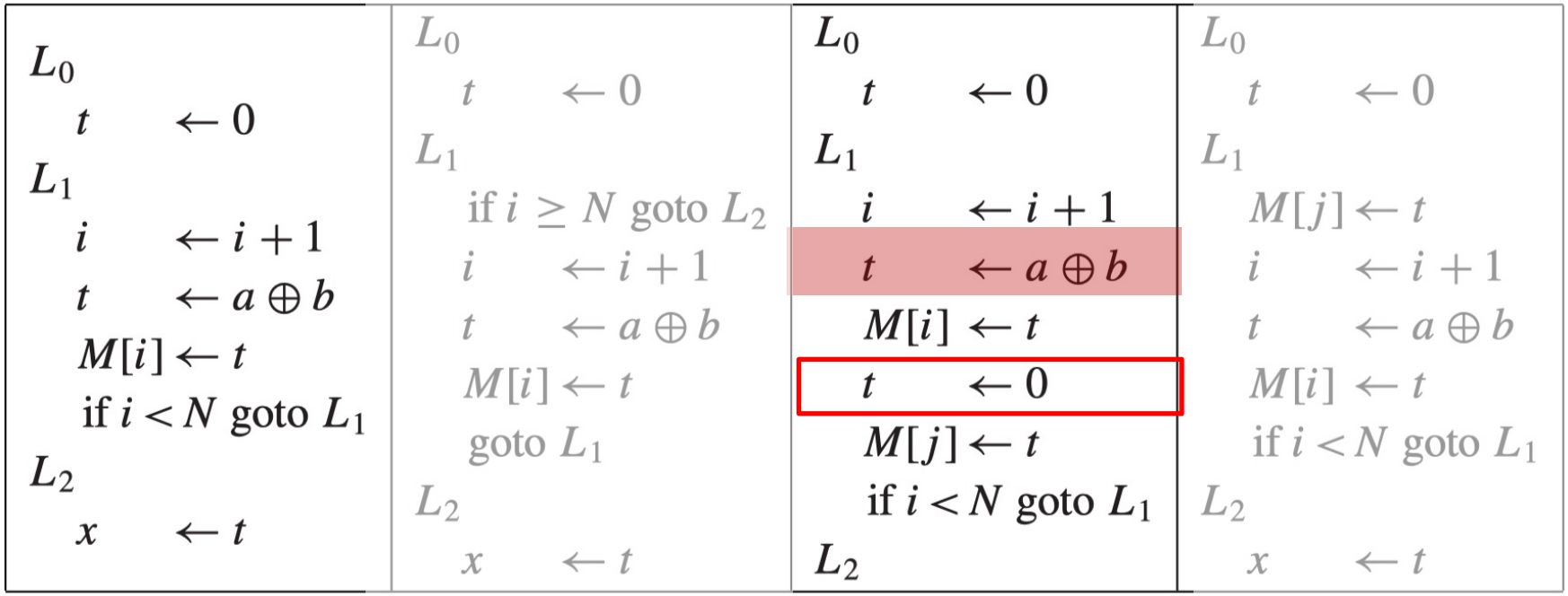


correct faster	incorrect not always execute the def

Example: Can We Hoist $d: t \leftarrow a \oplus b$?

2. and there is only one definition of t in the loop

The original loop had more than one definition of t, and the transformed program interleaves the assignments to t in a different way

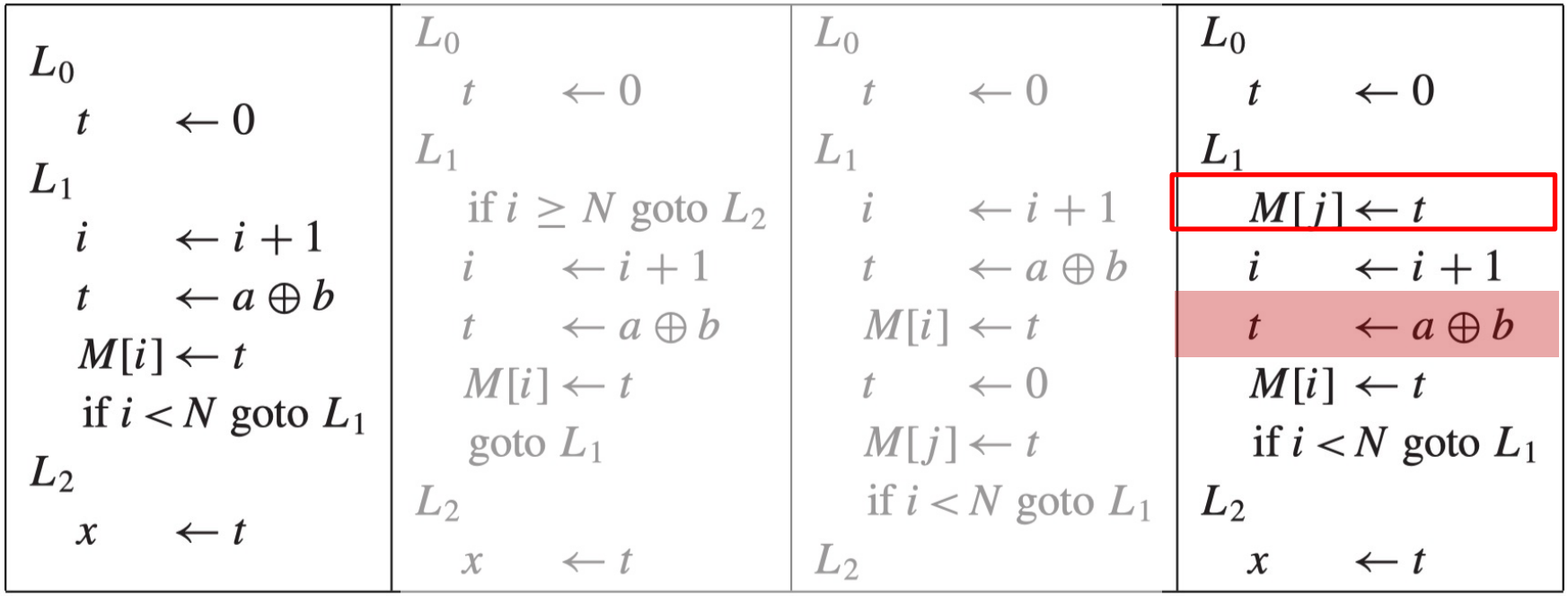


correct faster	incorrect not always execute the def	incorrect more than one def of t

Example: Can We Hoist *d: t ← a ⊕ b*?

3. and *t* is not live-out of the loop preheader

There is a use of *t* before the loop-invariant definition, so after hoisting, this use will have the wrong value on the first iteration of the loop.



correct faster	incorrect not always execute the def	incorrect more than one def of t	incorrect a use of t before the def

Thank you all for your attention